

LAB 01 Haskell

Ejercicio 1:

1.

```
[10]: [1,2,3]++[4,5]
```

```
[1,2,3,4,5]
```

2.

```
[11]: :t True
      :t "Hola mundo"
      :t [1,2,3]
```

```
True :: Bool
```

```
"Hola mundo" :: String
```

```
[1,2,3] :: forall {a}. Num a => [a]
```

Pregunta teórica: ¿Qué significa que Haskell sea un lenguaje tipado estéticamente y evaluado perezosamente?

RTA: Haskell es un lenguaje tipado estéticamente porque en compilación se verifican los tipos, y gracias a la inferencia de tipos casi nunca hay que escribirlos. Que sea evaluado perezosamente significa que no ejecuta funciones ni calcula resultados hasta que se ve forzado a hacerlo.

3.

A

```
[8]: doble :: Int -> Int
     doble x = x * 2
     doble 5
```

```
10
```

B. --

```
[9]: esMayorEdad :: Int -> Bool
     esMayorEdad x = x>=18
     esMayorEdad 19
```

True

C.

```
13]: areaCirculo :: Float -> Float
     areaCirculo radio = pi * radio^2
     areaCirculo 30
```

2827.4333

Pregunta de reflexión: ¿Qué diferencias encuentras entre los tipos Int y Float?

RTA: Int se usa para enteros positivos y negativos, float se usa para números de punto flotante.

Ejercicio 2:

1.

```
[1]: head [10,20,30,40]
     tail [10,20,30,40]
     take 3 [1..10]
     drop 4 [1..10]
     length [1,2,3,4,5]
```

10
[20,30,40]
[1,2,3]
[5,6,7,8,9,10]
5

Pregunta: ¿Qué hace la expresión [1..10] y cómo se relaciona con la evaluación perezosa?

RTA: La expresión [1..10] devuelve una lista de números enteros positivos del número 1 hasta el número 10, se relaciona con la evaluación perezosa porque cuando se le pide una lista como esta a haskell este solo va a calcular hasta el dígito que se está solicitando, en el ejemplo take 3, a pesar de que la lista está hasta el 10, haskell solo calcula la lista hasta el 3, como ya encontró lo que se le solicitó entonces no calcula más números en la lista.

A.

```
primerElemento :: [a] -> a
primerElemento = head
primerElemento [1,2,3,4]
```

1

B.

```
productoLista :: [Int] -> Int
productoLista = product
productoLista [1,2,3,4,8]
```

192

C.

```
eliminarNegativos :: [Int] -> [Int]
eliminarNegativos [] = []
eliminarNegativos (x:xs) = if x<0 then eliminarNegativos xs else x:eliminarNegativos xs
eliminarNegativos [1,2,3,-1,-2,-3,4]
```

[1,2,3,4]

D.

```
invertir :: [a] -> [a]
invertir [] = []
invertir (x:xs) = invertir xs ++ [x]
invertir [1,2,3,4,5]

[5,4,3,2,1]
```

Ejercicio 3:

a)

```
sumaCuadrados :: Int -> Int -> Int

sumaCuadrados x y = x^2 + y^2
sumaCuadrados 8 6
```

100

b)

```
dobleLista :: [Int] -> [Int]
dobleLista [] = []
xsdobleListas = map(*2)
dobleListas [4,6,8,14]

[8,12,16,28]
```

2.

a)

```
clasificarNota :: Int -> String
clasificarNota x =
    if x < 3 then "Reprobado"
    else if x < 4 then "En recuperación"
    else "Aprobado"
```

b)

```
fibonacci :: Int -> Int
```

Ejercicio 4:

1.

a) resultado: 2,4,6,8,10

Pregunta de análisis: ¿Por qué [1..] no causa un ciclo infinito?

Porque se especifica que solo tome los 5 primeros de la lista, entonces no calcula mas numeros despues de tomar los primeros 5 de la lista.

2.

En este ejercicio x es mayor que 10, y por ende se multiplica y por 2, pero y vale 4 x 3 es decir 12 por 2, 24, x vale 12 y y 24

Ejercicio 5:

1. Definir funciones lambda:

a. Doble de un numero

```
(\x -> x * 2) 5
```

Avoid lambda using `infix`

Found: **Why Not:**

```
(\ x -> x * 2) (* 2)
```

10

b. Cubo de un numero

```
(\ x -> x ^ 3) 3
```

Redundant lambda

Found:

Why Not:

```
cubo = \ x -> x ^ 3cubo x = x ^ 3
```

Avoid lambda using `infix`

Found:

Why Not:

```
\ x -> x ^ 3(^ 3)
```

27

c. Numero dividido entre 2

```
(\ x -> x / 2) 10
```

Avoid lambda using `infix`

Found:

Why Not:

```
(\ x -> x / 2)(/ 2)
```

5.0

d. Si un num es positivo

```
(\x -> x >= 0) 5
```

Avoid lambda using `infix`

Found:

Why Not:

```
(\ x -> x >= 0)(>= 0)
```

True

e. Si un num pertenece al intervalo (10,20)

```
(\ x -> x >= 10 && x <= 20) 21
```

False

2. Funciones lambda con dos argumentos.

a. Suma dos num

```
(\ x y -> x + y) 12 21
```

Avoid lambda

Found: **Why Not:**

```
\ x y -> x + y(+)
```

33

b. Multiplicación dos num

```
(\x y -> x * y) (-2) (-2)
```

Avoid lambda

Found: **Why Not:**

```
\ x y -> x * y(*)
```

4

c. Potencia entre dos num.

```
(\x y -> x ^ y) (-2) (3)
```

Avoid lambda

Found: **Why Not:**

```
\ x y -> x ^ y(^)
```

Redundant bracket

Found:**Why Not:**

(3) 3

-8

d. Máximo entre dos números.

```
(\x y -> if x > y then x else y) (5.001) (5)
```

Use max

Found:

Why Not:

```
if x > y then x else y
```

Redundant bracket

Found: Why Not:

```
(5.001)5.001
```

Redundant bracket

Found:Why Not:

```
(5) 5
```

```
5.001
```

e. Definir si dos numeros son iguales.

```
(\x y -> x == y) (67)(67)
```

Avoid lambda

Found:

Why Not:

```
\ x y -> x == y(==)
```

Redundant bracket

Found:Why Not:

```
(67) 67
```

Redundant bracket

Found:Why Not:

```
(67) 67
```

```
True
```


Retrospectiva

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?
(Horas/Hombre)

Matias Carballo: 3 horas

Samuel Sanchez: 3 horas

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

Terminado, porque completamos todos los ejercicios

3. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Aprender sintaxis y operaciones ya que ah medida que se resolvían ejercicios iniciales los proximos se sentian mas comprensibles a tener mas clara la sintaxis.

4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

No conseguimos entrar a la plataforma de haskell para desarrollar los ejercicios, le pedimos yuda al profesor.

5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Dividirnos los ejercicios y entender como los resolvimos, para mejores resultados seria practicar ejercicios adicionales entes de un nuevo lab para entrar con mas claridad.

6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

La guia de haskell y apuntes de clase.